TrustCore-v1

Final Report — Two-Month Internship

Taehee Kim
Seoul National University

July – August 2026

1 Summary

TrustCore-v1 is a hardware root of trust for naval and defence equipment. It sits between a host and its firmware: before a device is allowed to boot, the chip hashes the firmware image and the host compares that digest against a known-good reference. It also authenticates device identity and transmitted data using Ascon-128. The intended deployment is shipboard — ECDIS, AIS / GNSS / radar, gateways, PLCs, propulsion and steering controllers, VDRs, autonomous navigation computers.

The plan was to tape it out on an ETRI multi-project-wafer shuttle in 0.5 µm CMOS, on a 1 mm^2 die, at 25 MHz, with an SPI slave interface. That did not happen, and the reason is unambiguous.

> **Genus returned 8.49 mm^2 of cell area against a 1 mm^2 die budget — 8.5× over.** The pre-synthesis estimate in the project specification was 14 000–22 000 GE. The synthesised netlist is **39 333 GE**, roughly twice the top of that range. Every major block individually exceeds the entire die budget: SHA-256 is 3.77 mm^2, Ascon-128 is 2.30 mm^2, and the SPI buffer alone is 1.75 mm^2.

Timing was never the problem. The design closes at 150 MHz in 0.5 µm — six times the 25 MHz target. What killed it was state: 2 985 flip-flops occupying 55.7% of the total area.

The RTL was also wrong. Six defects were found, five confirmed by running the original code rather than reading it. One is a genuine cryptographic correctness bug: the Ascon-128 core produced a wrong authentication tag for *every* input while producing correct ciphertext. That would have shipped in silicon and would not have been noticed until someone tried to authenticate.

Rather than lose the work, the project pivoted to a ZCU106. The corrected RTL is instantiated *unmodified* as `chip_top` inside an FPGA-only harness, driven over AXI4-Lite by the Cortex-A53. That build runs on real hardware: the on-board run reported every vector in the run passing, with all eight user LEDs solid. It closes timing at 100 MHz with 5.868 ns of slack and burns 31 mW in the PL.

So the deliverable is not a chip. It is a corrected, verified crypto core, a measured hardware baseline, and a documented, quantified reason why the original target was never reachable.

The code, and where to look in this report

Everything described here — the RTL, the testbenches and Python golden reference, the Genus synthesis reports for all 13 frequency points, the constraints, and the Vivado / Vitis project — is public:

`https://github.com/tk685-cpu/trustcore`

The report is written so that each section answers one question. If you only want a specific answer, go straight to it.

If you want to know...	Read
What the chip was specified to do, and how a host talks to it	Section 2
Why the tape-out did not fit — the area numbers, block by block, and what I got wrong in the original estimate	Section 3
Why the project moved to an FPGA, and what that does and does not prove	Section 4
The six defects found in the original RTL, including the Ascon tag bug, and how each one was confirmed	Section 5
How correctness was established — known-answer tests, testbenches, and the run on real hardware	Section 6
The measured results: utilisation, timing, power, latency, and why the system is I/O bound	Section 7
How this compares against published SHA-256 and Ascon-128 implementations	Section 8
Known limitations, and what I would do next if the work continued	Section 9
How to clone the repository and re-run the simulations and synthesis yourself	Section 10

Section 10 also maps every directory in the repository to what it contains, so a reader who wants to go straight to the source can start there.

2 What the chip was supposed to be

A host sends a command frame over SPI, the chip runs either a SHA-256 hash or an Ascon-128 authenticated encryption, and the host reads 32 bytes of result back in a second frame.

Item	Specification
Purpose	Hardware root of trust: firmware integrity, device and data authentication
Deployment	Shipboard sensor and control modules on naval vessels
Process	0.5 µm CMOS, ETRI MPW shuttle
Cell library	`khu_etri05_stdcells` — 2-poly 3-metal, 5 V, typical 25 °C
Die budget	1 mm^2
System clock	25 MHz
SPI clock	5 MHz (host-driven)
Pins	8 — VDD, VSS, CLK, RST_N, SCLK, MOSI, MISO, CS_N
Algorithms	SHA-256 (FIPS 180-4), Ascon-128 v1.2 (NIST LWC winner)
Message limits	SHA-256 0–32 B, Ascon 0–16 B plaintext
Architecture	Iterative, 1 round per clock cycle, both cores

Table 1: TrustCore-v1 as specified. Ascon is the right algorithm choice here: it won the NIST lightweight cryptography competition and was designed for exactly this constraint envelope.

The intended flow, per the project specification: the host sends the firmware bytes over SPI with `CMD = SHA-256`; `crypto_fsm` routes the request to the on-chip core, waits, and returns the digest; the host compares against a known-good reference and decides whether to allow boot. Note that **the boot decision is not made on-chip** — the chip reports, the host decides. That is a design limitation worth stating plainly, because a root of trust that only advises is a weaker guarantee than one that gates.

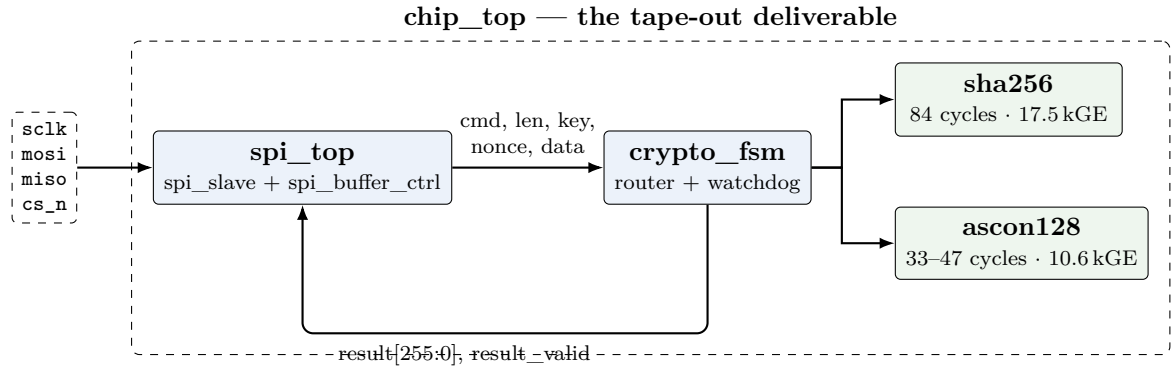

Figure 1: `chip_top`. Everything inside the dashed box was the ASIC deliverable, and is instantiated on the FPGA with no changes. Gate counts are from Genus.

3 Why the ASIC path did not close

3.1 The estimate, and the measurement

The project specification carried a rough pre-synthesis gate estimate. Genus was then run against the real ETRI library across a frequency sweep. The two do not agree.

Block	Spec estimate	Genus actual	Error
SHA-256 core	10 000–15 000 GE	17 455 GE	1.2–1.7×
Ascon-128 core	3 000–5 000 GE	10 627 GE	**2.1–3.5**×
SPI + buffer + router	1 000–2 000 GE	11 252 GE	**5.6–11**×
Security control FSM	∼350 GE	(folded into router)	—
Total	**14 000–22 000 GE**	**39 333 GE**	**1.8–2.8**×

Table 2: Pre-synthesis estimate against measured synthesis, at 35 MHz. Gate equivalents use the library's own `NAND2X1` at 216.0 µm^2. The estimate was not absurd — it was simply never validated, and the interface logic was underestimated by nearly an order of magnitude.

The single largest error is not in the cryptography. It is in the plumbing. `spi_buffer_ctrl` was estimated at a fraction of 1 000–2 000 GE and came back at **8 100 GE** — 79% the size of the entire Ascon core, for what is conceptually a byte shift register. The reason is visible in the port list: it holds a 128-bit key, a 128-bit nonce, 256 bits of data and a 256-bit result *simultaneously*, all in flip-flops, all at once.

3.2 Area against the die budget

Block	Cells	Area (µm^2)	mm^2	GE	vs 1 mm^2
`sha256`	8 658	3 770 298	3.770	17 455	**3.77**×
`ascon128`	6 227	2 295 423	2.295	10 627	**2.30**×
`spi_buffer_ctrl`	2 821	1 749 609	1.750	8 100	**1.75**×
`crypto_fsm` glue	1 066	608 000	0.608	2 815	0.61×
`spi_slave`	112	72 729	0.073	337	0.07×
`chip_top`	**18 845**	**8 496 018**	**8.496**	**39 333**	**8.50**×

Table 3: Genus `report_area`, `chip_top` at 35 MHz, typical corner. Note the `Net-Area` column in the raw report is zero and the wireload model is `<none>` — this is *cell area only.* A real placed and routed die would be larger still, before adding the pad ring.

Only `spi_slave`, at 0.073 mm^2, would have fit in the die budget. **Every other block in the design individually exceeds the entire 1 mm^2 slot** — the SHA-256 core by nearly 4× on its own.

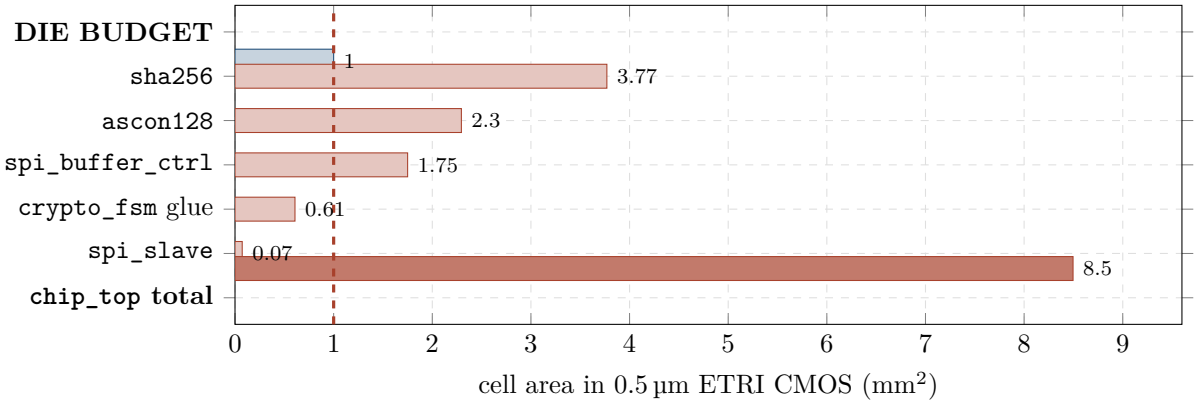

Figure 2: Measured area against the 1 mm^2 MPW slot (dashed line). Everything except `spi_slave` is over budget on its own.

3.3 Where the area went

Cell type	Instances	Area (µm^2)	Share
Sequential (`DFFSR`)	2 985	4 728 240	**55.7%**
Logic	12 180	3 205 431	37.8%
Inverters	3 636	533 880	6.3%
Buffers	83	22 752	0.3%
Total	18 884	8 490 303	100%

Table 4: `report_gates` at 50 MHz. A single `DFFSR` costs 1 584 µm^2, or 7.33 GE. The design is a register file with some arithmetic attached.

Flip-flops are the whole story. The architectural state is unavoidable — SHA-256 needs 8 working variables, 8 hash registers and a 16-word message schedule window (1 024 b total), Ascon needs a 320-bit permutation state — but the *interface* state was a choice. The 256-bit parallel input ports and the simultaneous key/nonce/data/result buffering exist only because the protocol was designed to hand each core a whole message at once.

3.4 Frequency, area and power

Two things follow from this table.

Timing was never the constraint. The design has 6× headroom at the target frequency. All of the effort that would normally go into closing timing was available to spend on area, and none of it would have been enough: the gap is 8.5×, not 10%.

Power is a second, independent problem. 0.51 W at 25 MHz is a lot for a part described as a lightweight root of trust. The cause is the library: `khu_etri05_stdcells` is characterised at a **5 V** nominal supply, and dynamic power scales with V^2. A 1 mm^2 die dissipating half a watt is also a thermal question, not just an energy one. Leakage is negligible (2.6 µW), as expected at this node.

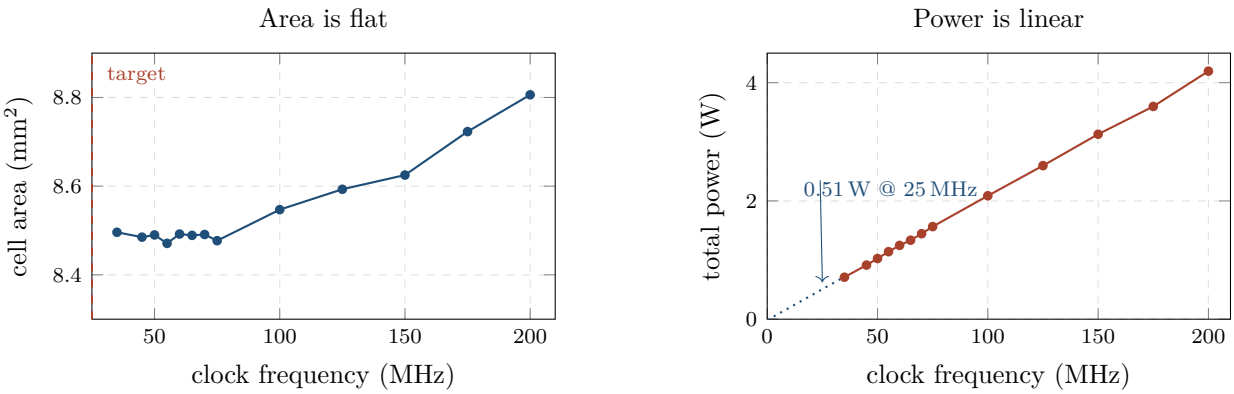

Figure 3: Genus PPA sweep, 13 frequency points. Area is essentially constant from 35 to 75 MHz and grows only 3.6% out to 200 MHz — confirming the design is state-bound, not timing-bound. Power fits $P = 20.9\,\text{mW/MHz} \times f - 16\,\text{mW}$ with $R^2 = 0.9998$, extrapolating to **0.51 W at the 25 MHz target**.

Freq (MHz)	Cells	Area (mm²)	Slack (ps)	Power (W)	Status
35	18 845	8.496	+12 344	0.710	met
45	18 715	8.485	+5 721	0.915	met
50	18 884	8.490	+3 838	1.026	met
55	18 763	8.471	+2 336	1.142	met
60	18 856	8.492	+820	1.248	met
65	18 948	8.489	+283	1.336	met
70	18 928	8.491	+26	1.446	met
75	18 901	8.477	+2	1.566	met
100	19 107	8.547	+103	2.087	met
125	19 207	8.593	+6	2.597	met
150	19 220	8.625	0	3.128	met (marginal)
175	19 560	8.723	−157	3.598	**violated**
200	20 040	8.806	−698	4.195	**violated**

Table 5: The full sweep. $F_{\max}$ **in 0.5 µm is 150 MHz** — six times the 25 MHz target. The critical path is in `u_sha256` at *every* frequency point, running from `round_reg` through the Σ/CSA network into `e_reg`.

3.5 On my own earlier estimate

Before the Genus reports were located, this report carried a bottom-up reconstruction from the FPGA netlist. It is worth recording how it did, because the failure mode is instructive.

	Reconstruction	Genus actual	
Flip-flops	3 032 (FPGA)	2 985 (`DFFSR`)	within 1.6%
Gate count	∼33 kGE	39 333 GE	16% low
µm² per GE	80–90 assumed	**216.0 measured**	**2.5× low**
Total area	∼4.4 mm²	**8.496 mm²**	1.9× low

Table 6: The logical estimate was close. The physical one was not.

The gate count was recoverable from the FPGA netlist to within 16%. The area was not, because I assumed a denser 0.5 µm library than this one actually is. `khu_etri05_stdcells` is a 2-poly 3-metal 5 V analog CMOS process — 216 µm² per NAND2 against the 80–90 typical of a digital-optimised 0.5 µm library. **The conclusion was right and the margin was understated by half.** Never scale area across processes without the actual library.

3.6 What should have happened

The project plan had six steps: architectural specs, verification, RTL design, synthesis, place and route, sign-off. Synthesis was step 4. The area problem was therefore discovered *after* the RTL was written and verified, which is the most expensive place to find it.

Three cheap checks, any one of which would have caught it before step 3:

1. **Count the architectural state and multiply.** 1 024 b (SHA) + 320 b (Ascon) + 768 b (SPI buffers) ≈ 2 100 flip-flops on paper. At the library's measured 1 584 µm^2 per `DFFSR` that is 3.3 mm^2 of *registers alone* — already 3× the die. This is arithmetic, not synthesis.
2. **Synthesise the two cores early**, against the real library, on day one of RTL. Genus takes minutes on a design this size.
3. **Sanity-check the estimate against published work.** Iterative SHA-256 is widely reported at 15–22 kGE; the spec assumed 10–15 kGE. Ascon-128 at one round per cycle is published at 7.9–9.4 kGE [3, 6]; the spec assumed 3–5 kGE. Both estimates were below every published figure.

There is a design answer as well as a planning one. A 1 mm^2 0.5 µm die can hold Ascon-128 — but only if the wide parallel ports are replaced with a byte-serial interface and SHA-256 is dropped. Removing SHA-256 (3.77 mm^2) and the buffer controller's parallel storage (1.75 mm^2) takes 5.5 of the 8.5 mm^2 out at a stroke. That is a different chip, and it is the chip that would have taped out.

4 The pivot to FPGA

Once the die was clearly out of reach, the work was redirected to producing a verified core plus a hardware platform that proves it. The RTL defects are real regardless of target; an FPGA gives a measurable baseline the ASIC path was never going to produce; and timing closure at 100 MHz is strictly harder than at the 25 MHz ASIC target.

The board was a ZCU106 (XCZU7EV-FFVC1156-2-E), Vivado 2025.1.

4.1 Two decisions worth defending

A custom AXI4-Lite peripheral, not AXI Quad SPI. The obvious route is to drop in Xilinx's AXI Quad SPI IP as the master. I wrote a small custom peripheral instead, for one reason: the whole PS-to-PL path can then be simulated end to end against a real AXI4-Lite bus-functional model, and IP GUI settings cannot. AXI Quad SPI has a build-time frequency ratio, a slave-select register, FIFO behaviour and an `XSpi` driver with real subtleties around manual slave select — every one of those is a place to lose a day during bring-up. The custom peripheral also has a *runtime*-settable clock divider, which is what made the SCLK margin sweep possible on hardware. The cost is about 300 lines of Verilog that we own.

One clock domain. The AXI interface, the SPI master and `chip_top` all run on `pl_clk0`. No clock domain crossing anywhere, the SPI path is an ordinary register-to-register path that static timing analysis fully covers, and there are no I/O timing constraints to get wrong. Behaviour is identical to a 25 MHz clock because the design is fully synchronous — what matters is the SCLK-to-clock *ratio*, not the absolute frequency.

5 What was wrong with the RTL

Six defects. Five were confirmed by running the original code in simulation, not by reading it.

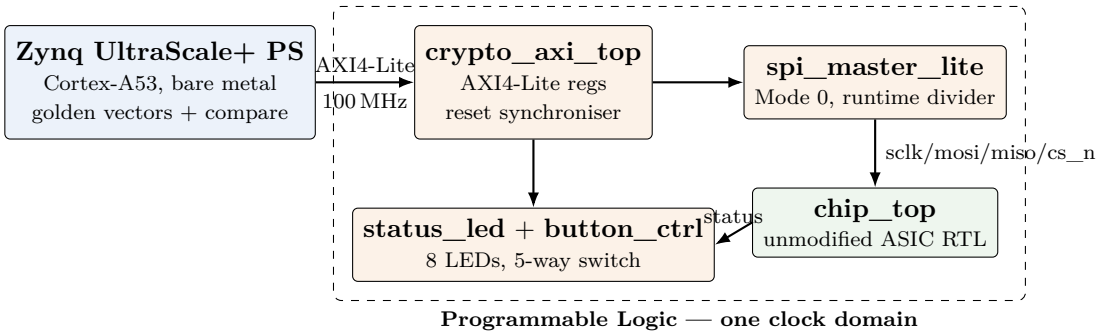

Figure 4: The ZCU106 validation platform. Orange blocks are FPGA-only scaffolding and are not intended to tape out; green is the ASIC deliverable, instantiated with no changes.

5.1 Ascon-128 produced the wrong authentication tag

Critical. Ascon requires the plaintext to be padded with `0x80` followed by zeros up to a multiple of the 8-byte rate, and critically *a full padding block is appended even when the length is already a multiple of 8*. The original core skipped the padding block entirely.

The ciphertext still came out correct, because the padding block contributes no ciphertext bytes. But the padding block is absorbed into the state before finalisation, so the tag was wrong for every single input:

```
pt_len=0   tag got 2cd6a332f04aaf580df298a559aa76a9
           tag exp e355159f292911f794cb1432a0103a8a   <- NIST LWC KAT

pt_len=16  ct  got bc820dbdf7a4631c5b29884ad69175c3   <- ciphertext correct
           tag got f949e005f483688490270dd7033dfba5
           tag exp f58e28436dd71556d58dfa56ac890beb   <- tag wrong
```

A wrong tag is the worst kind of failure here: encryption looks like it works, and the fault only surfaces when someone tries to decrypt or authenticate. For a part whose stated job is authenticating device identity and transmitted data on naval equipment, this is the defect that matters most.

Fixed by implementing the padding inside the core, including the ciphertext truncation mask so pad bytes never leak into the output. It now passes all 37 vectors covering every plaintext length 0 to 32, plus associated-data cases.

5.2 Result transmission never terminated

`byte_idx` in `spi_buffer_ctrl` was `reg [4:0]`, so it counted 0 to 31. The transmit loop exited on `byte_idx >= result_len_latch` with `result_len_latch` always 32. A 5-bit counter can never reach 32, so the comparison was never true. The FSM streamed 32 correct bytes, then wrapped to 0 and kept streaming zeros forever, never returning to idle. The observable symptom: the first command works, and every command after it is silently ignored.

5.3 MISO was one bit early from the second byte onward

The controller asserts `tx_load` a few system clocks after a byte completes, which lands *before* the SCLK falling edge that ends that bit period. The original code applied the load straight to the shift register, so that falling edge shifted the freshly loaded byte one more time:

```
got 63 1b 9b 53 cd 89 ...
exp 63 0d cd 29 66 c4 ...
     ^  ^^ every byte after the first is shifted left by one bit
```

This is the kind of bug that passes a core-level testbench and only appears once you drive real SPI traffic. Fixed by staging the incoming byte in `tx_hold`/`tx_pending` and swapping it into the shifter *on* the falling edge, which is the actual byte boundary.

5.4 The other three

- **Message alignment mismatch.** `spi_buffer_ctrl` accumulated bytes by shifting into the LSB end, leaving the message right-aligned; both cores expect it left-aligned. The two only agree when `DATA_LEN` is exactly 32, so any shorter message hashed the wrong block — which for a firmware-integrity part means the wrong digest.
- **Zero-length input hung the receive loop.** `S_RX_DATA` exited on `byte_idx == data_len_out - 1`; with `data_len_out` zero that is `0xFF`, never reached.
- **Unrecognised commands still ran SHA-256.** A request with neither command bit set fell into the SHA branch and returned a plausible-looking wrong answer. Commands and lengths are now validated and raise `err`.

5.5 Also corrected

- **The documented SCLK limit was wrong, and the spec's SPI clock is not achievable.** The header claimed $f_{\text{clk}}/2$. Sweeping the rate in simulation, the design passes at $f_{\text{clk}}/6$ and fails at $f_{\text{clk}}/4$; the documented limit is now $f_{\text{clk}}/8$. **The project specification calls for a 5 MHz SPI clock at a 25 MHz system clock** — that is $f_{\text{clk}}/5$, above the safe limit and inside the measured failure band. Either the system clock has to rise or the SPI clock has to drop. This was not previously flagged anywhere.
- **Frame abort recovery.** A frame ending early used to leave the FSM stuck until reset. Receive states now abort cleanly back to idle, which matters during bring-up: one glitch no longer means a board reset.
- **Watchdog.** If a core never asserted `done` the chip locked up forever. `crypto_fsm` now bounds the wait and returns an error result.
- **Reset domain crossing.** `rst_n` was used as an asynchronous reset in some blocks and a synchronous reset in others, so different parts of the design could leave reset on different clock edges — intermittent start-up failures that are very hard to reproduce. The FPGA wrapper now contains a proper async-assert / sync-release reset synchroniser.

6 Verification

The SCLK margin sweep. $f_{\text{clk}}/16$, /10, /8 and /6 pass; /4 and /2 fail. Edge detection compares adjacent synchroniser stages, and the TX byte handover needs three system clocks between a rising edge and the following falling edge, so the limit is structural rather than arbitrary.

On hardware. The bitstream was programmed onto the ZCU106 and the bare-metal application launched on `psu_cortexa53_0` from Vitis. The first thing it does is read the ID register at offset `0x00`, which returns `0x5AA5C0DE`: if that reads back correctly then the bitstream, clock, reset, address map and AXI path are all working, and any later failure is a logic problem rather than an infrastructure problem. It then pushes known vectors through the real SPI pins and compares on the A53. Every vector in the run passed and all eight LEDs went solid.

The application holds 200 SHA-256 and 200 Ascon-128 known-answer vectors and draws a random subset per button press, seeded from the CPU cycle counter at the moment of the press, plus three

Test	What it covers	Result
`button_unit`	debounce, both-press window, hold, glitch rejection	pass
`sha256_unit`	36 vectors, every length 0–32, against Python `hashlib`	pass
`ascon_unit`	37 vectors, every length 0–32 plus associated-data cases	pass
`chip_e2e`	29 vectors through the real SPI pins, plus rejection and abort	pass
`axi_stack`	17 checks through an AXI4-Lite bus-functional model	pass
`sclk_margin`	SCLK rate sweep to find the actual failure point	characterised
lint	Verilator `-Wall` on `chip_top` and `crypto_axi_top`	clean
C build	`gcc -Wall -Wextra` on the Vitis application	clean
Hardware	ZCU106, Vitis on `psu_cortexa53_0`	**all pass**

Table 7: The full suite, re-run for this report. The Ascon golden reference is validated against the published NIST LWC known-answer test: key and nonce `000102...0F`, empty AD, empty plaintext gives tag `E355159F292911F794CB1432A0103A8A`.

negative tests (bad command, oversize length, recovery). So a single run exercises a different sample each time rather than the same fixed list — which is the right behaviour for catching input-dependent faults, but it does mean the pass count printed over UART depends on the build's `SHA_PER_RUN` / `ASC_PER_RUN` settings rather than being a fixed figure.

The eight user LEDs are driven as one bar rather than a red/green pair, because the ZCU106's user LEDs are single-colour. Idle is a slow pulse (~0.38 Hz), running is a fast even blink (~6 Hz), pass is solid on, fail is dark. Anything moving means "no verdict yet"; anything static is the answer. Idle deliberately pulses rather than sitting dark — otherwise a board waiting at the menu, a crashed application and a bitstream that never programmed would all look identical to a failing test.

7 Measured FPGA results

7.1 Resource utilisation and timing

Module	LUT	FF	CARRY8	WNS (ns)	$F_{\max}$ (MHz)
`sha256`	1 188	1 302	52	5.832	240
`ascon128`	1 341	728	0	7.219	360
`crypto_fsm`	2 603	2 179	52	6.011	251
`spi_top`	608	862	0	7.914	479
`chip_top` (ASIC deliverable)	3 232	3 032	52	5.740	235
Full system, placed & routed	3 923	3 885	71	5.868	**242**

Table 8: XCZU7EV-FFVC1156-2-E, Vivado 2025.1. Rows 1–5 are out-of-context synthesis, place and route with a 10 ns constraint. The last row is the complete block design including the AXI SmartConnect, LEDs and buttons. $F_{\max}$ is $1/(T - \mathrm{WNS})$, which is optimistic: the tools stop optimising once the constraint is met.

Zero BRAM, zero URAM, zero DSP. 781 CLBs, 185 unique control sets. The FPGA's 3 032 flip-flops for `chip_top` agree with Genus's 2 985 to within 1.6%, which is a useful cross-check that the two flows are building the same design.

7.2 Power

The FPGA's 31 mW and the ASIC's 0.51 W are not directly comparable — 16 nm FinFET at 0.85 V against 0.5 µm at 5 V is a 30-year gap — but the contrast is worth stating, because it is the

Component	Power	Note
`crypto_axi_top` (all PL logic)	**31 mW**	the number that means anything
AXI SmartConnect	5 mW	interconnect overhead
PS8 (Cortex-A53 subsystem)	2 656 mW	fixed Vivado estimate for the MPSoC
Device static	692 mW	593 mW PL, 98 mW PS
Total on-chip	3 386 mW	

Table 9: `report_power` on the routed design, 100 MHz, default switching activity. Confidence level: low — no SAIF was back-annotated, so treat 31 mW as an upper-bound estimate rather than a measurement. The headline 3.4 W is almost entirely the hard processor system and the static leakage of a large FPGA; neither has anything to do with the crypto design.

strongest practical argument for not reviving the 0.5 µm target.

7.3 Latency, measured in simulation

Core	Case	Cycles	@ 25 MHz	@ 100 MHz
SHA-256	any length 0–32 B	84	3.36 µs	0.84 µs
Ascon-128	`pt_len`=0	33	1.32 µs	0.33 µs
Ascon-128	`pt_len`=8	40	1.60 µs	0.40 µs
Ascon-128	`pt_len`=16	47	1.88 µs	0.47 µs
Ascon-128	`pt_len`=16, `ad_len`=16	71	2.84 µs	0.71 µs

Table 10: Cycle counts measured with a dedicated testbench, counting from the `start` pulse to `done`. Ascon costs a flat 7 cycles per additional 64-bit rate block (1 absorb + 6 permutation rounds). SHA-256 breaks down as 1 (pad) + 16 (schedule load) + 64 (compress) + 2 (output).

7.4 The system is I/O bound, not compute bound

This is the most important measurement in the report and it was not in the original specification.

At the ASIC target — 25 MHz system clock, SCLK capped at $f_{clk}/8 = 3.125$ MHz — one SPI byte takes 2.56 µs. A 32-byte SHA-256 request needs a 34-byte command frame and a 32-byte readback frame.

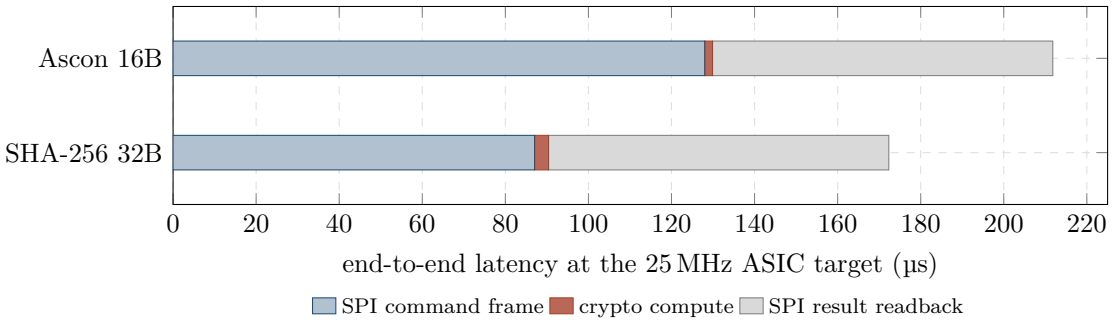

Figure 5: Where the time actually goes. The red slice is the entire reason the chip exists. It is 1.9% of a SHA-256 transaction and 0.9% of an Ascon transaction.

The cores are starved by factors of roughly 100 and 380. For a firmware-integrity part this compounds: hashing a 1 MB firmware image 32 bytes at a time, at 172 µs per transaction, takes about **5.6 seconds** — and the SHA-256 core is idle for 98% of it.

Operation	Total	Compute share	Effective	Core capability
SHA-256, 32 B	172.3 µs	1.9%	1.49 Mbps	152 Mbps
Ascon-128, 16 B	211.8 µs	0.9%	0.60 Mbps	229 Mbps

Table 11: At 25 MHz with SCLK at $f_{\text{clk}}/8$. "Core capability" is the sustained rate the core could deliver if data arrived instantly.

8 Comparison with published implementations

8.1 SHA-256

Work	Device	LUT	FF	Freq	Cyc/blk	Thr.
This work	XCZU7EV	**1 188**	**1 302**	240 MHz	**84**	1.46 Gbps
This work	0.5 µm ETRI	17 455 GE		25 MHz	84	0.15 Gbps
secworks/sha256 [1]	Artix-7 xc7a200t	2 471	1 930	108 MHz	66	0.84 Gbps
secworks/sha256 [1]	Spartan-6	2 012	1 929	70 MHz	66	0.54 Gbps
MDPI Sensors 2024, 1 core [2]	Virtex-6	6 730	794	12.7 MHz	65	0.10 Gbps
Ref. [38] in [2]	Artix-7	1 310	881	141.8 MHz	—	1.40 Gbps
Unfolded, in [2]	Arria II GX	1 215	871	—	32	2.43 Gbps
secworks/sha256 [1]	40 nm LP ASIC	14 200 µm^2		250 MHz	66	1.94 Gbps

Table 12: Throughput is $512 \times f/\text{cycles}$ per block. Frequency comparisons across device families are not meaningful; **cycles per block** is the number to read.

Where we stand. 17.5 kGE sits inside the published 15–22 kGE range for an iterative SHA-256, and 1 188 LUT / 1 302 FF is at the small end of the FPGA field. But this is partly an unfair comparison: our core handles a *single* 512-bit block with a fixed 256-bit input port and no streaming, so it does not carry the multi-block control a general-purpose core needs. A core that cannot hash more than 32 bytes is not really comparable to one that can hash a file.

Where we are behind: 84 cycles per block against a field of 65–66. That is 27% slower for no architectural reason. Our core loads all 16 message-schedule words serially before starting compression, when the standard approach folds the load into the first 16 compression rounds — $W[0..15]$ are consumed one per round anyway. Fixing this brings us to ~67 cycles and costs no area. Given that the critical path also lives in this core at every frequency point (Table 5), it is the obvious place to spend effort.

8.2 Ascon-128

Where we stand. 10 627 GE against Groß's 9 420 GE for the same 1-round-per-cycle architecture [3] and 7 891 GE for an area-optimised design [6]. We are 13% larger than the CAESAR-API reference and 35% larger than the best published — respectable, but not best-in-class, and the gap is almost certainly the 256-bit `ad` and `plaintext` input ports, which hold data the core consumes 64 bits at a time. On FPGA, 1 341 LUT is smaller than every design in the table, and it closes at 360 MHz, the fastest block in the design by a wide margin.

Where we are behind: rounds per cycle. Every high-performance design in the literature unrolls. Malal's 6-round unroll [4] finishes a 64-bit block in 2 cycles instead of 7, and Groß's ASIC data shows the trade curve directly: $1 \to 2$ rounds costs 38% more area for 73% more throughput. For a 25 MHz ASIC with 6× timing headroom that was the wrong trade — but on the FPGA it is a straightforward win.

Work	Platform	Rnd/cyc	LUT	FF	Area / Thr.
This work	XCZU7EV	1	**1 341**	**728**	3.28 Gbps @ 360 MHz
This work	0.5 µm ETRI	1	**10 627 GE**		0.23 Gbps @ 25 MHz
Malal [4]	Artix-7	6	2 957	595	3.54 Gbps @ 55 MHz
Malal [4]	Kintex-7	6	2 958	595	5.93 Gbps @ 93 MHz
Malal [4]	Spartan-7	6	2 957	595	3.44 Gbps @ 54 MHz
As tabulated in [5]	Artix-7	—	1 756	912	0.38 Gbps @ 55 MHz
Groß (CAESAR HW API) [3]	ASIC	1	9 420 GE		4.89 Gbps
Groß (CAESAR HW API) [3]	ASIC	2	12 989 GE		8.48 Gbps
Groß (CAESAR HW API) [3]	ASIC	6	27 280 GE		12.26 Gbps
Optimised IoT design [6]	ASIC	1	7 891 GE		—

Table 13: Our Ascon core is a 1-round-per-cycle iterative design, so the 1-round ASIC rows are the fair comparison.

9 What still needs work

Ordered by value per unit of effort.

1. Overlap the SHA-256 message schedule with compression. 84 → ~67 cycles, a 20% latency cut, at no area cost. The 16 schedule words are consumed one per round; there is no reason to load them in a separate 16-cycle phase first. *Effort: half a day. Risk: low — the 36-vector unit test catches any mistake immediately.*

2. Widen the host interface, or the chip is pointless. The measurements in Section 7.4 show the SPI link is 98–99% of every transaction. Three options, in increasing order of work:

- Raise SCLK. The measured failure point is $f_{\text{clk}}/4$; the documented limit is $f_{\text{clk}}/8$. Restructuring the TX byte handover so it does not need three system clocks between edges would make $f_{\text{clk}}/4$ safe — a straight 2×, and it would also make the specification's 5 MHz SPI clock legal.
- Use quad SPI, or a parallel bus. 4× for the data phases.
- Overlap. Nothing today lets the host stream the next command while the previous result is being read. Double-buffering the result register would hide the entire readback frame behind the next command.

Effort: 1–3 days depending on option. Risk: medium — the SPI timing is where the original bugs lived, so the margin sweep must be re-run.

3. Remove the 32-byte ceiling. Both cores are single-block. SHA-256 cannot hash anything longer than 32 bytes, which for a firmware-integrity part is close to disqualifying — real firmware images are megabytes. Ascon is capped at 16 bytes of plaintext by the 256-bit result bus, not by the core. Multi-block streaming means feeding the core a block at a time and keeping the chaining value; for SHA-256 this is mostly control logic, and the compression function does not change. *Effort: 3–5 days. Risk: medium.*

4. Move the boot decision on-chip. Today the chip returns a digest and the host compares it. That means the comparison — the actual security decision — happens in software the chip is supposed to be protecting. Storing a reference digest in on-chip OTP and driving a `boot_ok`/`boot_error` pin directly would close that gap, and it is already drawn that way in the project's own architecture diagram. *Effort: 2–3 days plus an OTP macro. Risk: medium.*

5. Expose associated data. `ad_len` is tied to zero in the SPI protocol. The core implements AD correctly and it is covered by the unit tests, so this is a protocol change, not a core change. Without it the Ascon interface is not really AEAD, only AE. *Effort: 1 day. Risk: low.*

6. Add Ascon decryption and tag verification. Encryption only, today. An AEAD core that cannot verify is half a core, and constant-time tag comparison in hardware is straightforward.

Effort: 2–3 days.

7. Side-channel resistance — nothing has been done here. This is a key-handling part for defence equipment with an unprotected datapath. The key sits in a plain register and the permutation is unmasked. Published protected Ascon designs run around 41 kGE against 9 kGE unprotected [5], a 4.5× area penalty, so this is a design-level decision that has to be made up front, not retrofitted. *Nobody should ship this against a physical-access threat model as it stands.*

8. If the ASIC is revived, re-scope before writing any RTL. Table 3 says what to cut. Dropping SHA-256 removes 3.77 mm^2; replacing the parallel input ports with a byte-serial interface removes most of `spi_buffer_ctrl`'s 1.75 mm^2. That takes 8.5 mm^2 down toward 3 mm^2 — still over a 1 mm^2 slot, so either the die grows or a newer node is used. At 0.5 µm and 216 $µm^2$ per gate, 1 mm^2 buys about 4 600 gates. That is the real budget, and it should be the first number in the next specification.

9. Housekeeping. The Vivado build script (`scripts/build_zcu106.tcl`) has never been executed against a real Vivado installation — the manual block-design flow is the reliable path. All pin assignments come from the ZCU106 Rev1.0 master XDC; a different board revision needs re-checking, because wrong pins still build and the LEDs simply never light. Genus was also run without a wireload model (`Net-Area = 0`), so the 8.5 mm^2 is a floor, not a final number — a physical-aware run with the supplied LEF and QRC tech files would give the real figure.

10 Reproducing this

All sources are at `https://github.com/tk685-cpu/trustcore` (the working copy on the lab machine is `~/Desktop/trustcore_vivado`). The layout of the repository is:

```
rtl/                  ASIC deliverable -- instantiated unmodified
  chip_top.v          top level, status pins
  spi_top.v spi_slave.v spi_buffer_ctrl.v
  crypto_fsm.v        command validation, watchdog
  sha256.v            iterative, 84 cycles/block
  ascon128.v          spec-compliant padding
rtl_fpga/             FPGA-only scaffolding -- not for tape-out
  crypto_axi_top.v    AXI4-Lite peripheral, reset synchroniser
  spi_master_lite.v   SPI Mode 0 master, runtime divider
  status_led.v button_ctrl.v
synthesis/            Genus flow
  genus.tcl constraints.sdc run_sweep.sh
  reports/            13 frequency points, area/gates/power/timing
  netlist/chip_top_genus.v
lib/                  khu_etri05_stdcells.lib, LEF, GDS2, QRC tech files
constraints/zcu106_crypto.xdc
scripts/run_sim.sh    the whole verification suite
vitis/main.c          bare-metal validation application
sim/                  testbenches + Python golden reference
plot/plot_ppa.py      the PPA sweep charts
```

Simulation: `cd scripts && ./run_sim.sh`, requires `iverilog`, `python3`, optionally `verilator`. Synthesis: `cd synthesis && ./run_sweep.sh`, requires Genus and the ETRI library.

> On the lab machine the Xilinx tools put an old `libstdc++` on `LD_LIBRARY_PATH`, which breaks `iverilog` with `GLIBCXX_3.4.32 not found`. Run `env -u LD_LIBRARY_PATH ./run_sim.sh` instead.

References

References

[1] J. Strömbergson, *secworks/sha256 — Hardware implementation of the SHA-256 cryptographic hash function*. FPGA and 40 nm ASIC results in the project README. `https://github.com/secworks/sha256`

[2] *SHA-256 Hardware Proposal for IoT Devices in the Blockchain Context*, Sensors 24(12):3908, MDPI, 2024. Comparison table of SHA-256 FPGA implementations 2002–2021. `https://www.mdpi.com/1424-8220/24/12/3908`

[3] *Ascon — Implementations*, Institute of Applied Information Processing and Communications, TU Graz. CAESAR Hardware API ASIC results by H. Groß. `https://ascon.isec.tugraz.at/implementations.html`

[4] A. Malal, *ascon-vhdl*, and *High-Performance FPGA Implementations of Lightweight ASCON-128 and ASCON-128a with Enhanced Throughput-to-Area Efficiency*, IACR ePrint 2025/825. `https://github.com/AhmetMALAL/ascon-vhdl`, `https://eprint.iacr.org/2025/825`

[5] J. Kaur, A. Cintas Canto, M. Mozaffari Kermani, R. Azarderakhsh, *A Comprehensive Survey on the Implementations, Attacks, and Countermeasures of the Current NIST Lightweight Cryptography Standard*, arXiv:2304.06222, 2023. `https://arxiv.org/abs/2304.06222`

[6] *An Optimized Hardware Implementation of ASCON Cipher for IoT Applications*, Springer, 2024. `https://link.springer.com/chapter/10.1007/978-981-97-3756-7_22`

[7] NIST Lightweight Cryptography project, Ascon known-answer test vectors. `https://csrc.nist.gov/projects/lightweight-cryptography`